Sculpting Latent Space

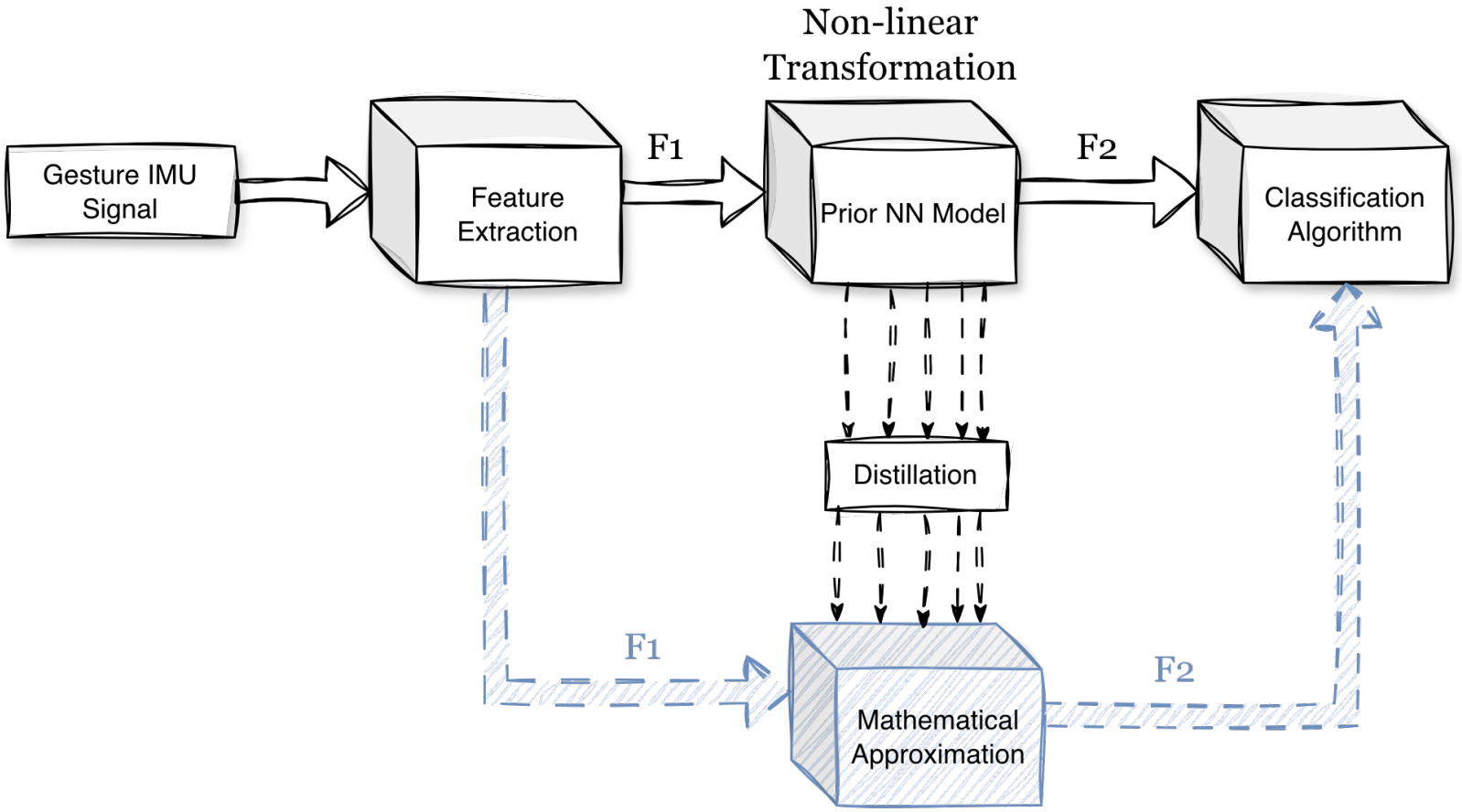
Introduction

In attempting to resolve the high false-positive rates in my gesture classification models, I began to question the standard "end-to-end" approach. **Instead of moving directly from input to classification, why not solve the problem within the latent space itself?**

Within this latent space, similar signals naturally form clusters, making classification theoretically and visually straightforward. However, static models are rigid; a new sample that "truly" belongs to a class may be mapped to the wrong region because the model's internal logic is frozen. To fix this, I propose a system that updates the latent mapping in real-time without requiring a full retraining of the neural network.

In this framework, the pre-trained model serves as a "prior": a foundational guide for converting gesture signals into latent embeddings. My approach explores how to mathematically approximate this model's internal transformation and then refine that approximation as new samples arrive. This approach relies on the hypothesis that the mapping is a reducible mathematical expression with smooth boundaries between classes, allowing for a more transparent, adaptive, and "living" latent space.

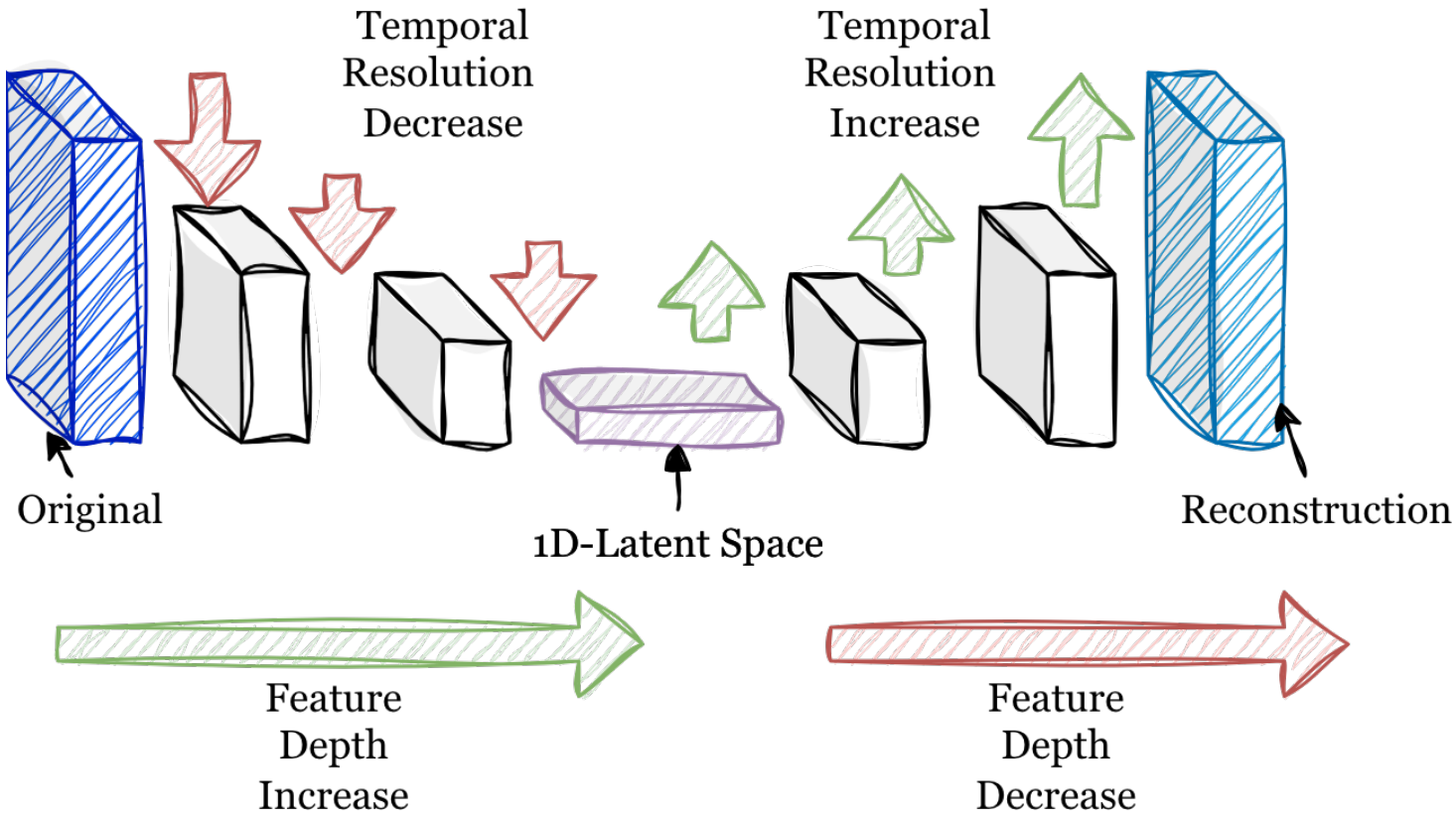
General Architecture



F1 → Reconstruction Features (nature of the signal itself)

F2 → Latent Features (forming clusters)

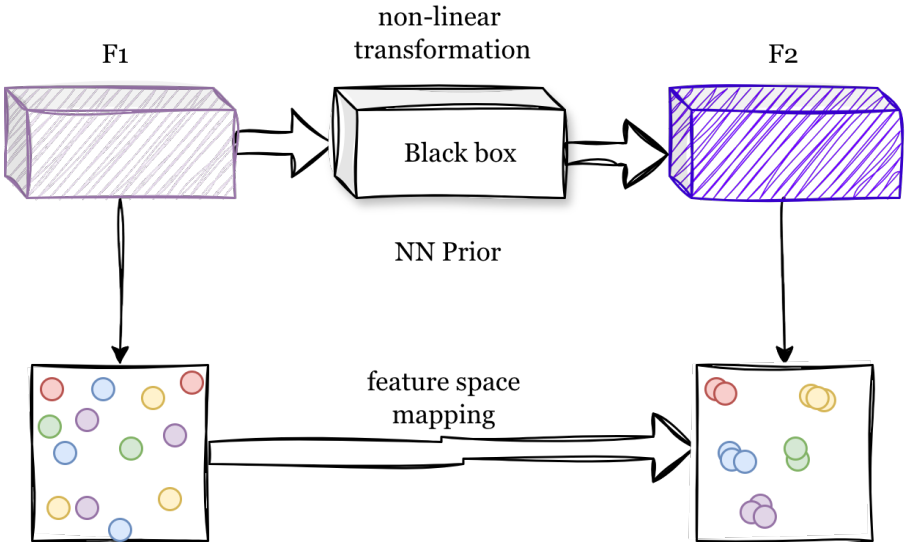
Feature Extraction Architecture

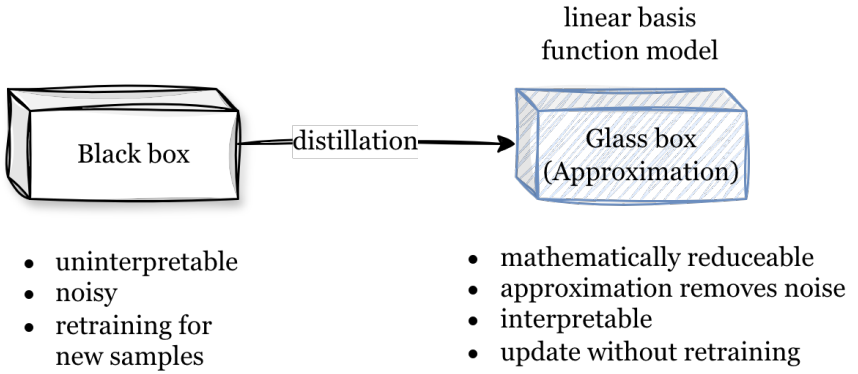


The model compresses raw IMU signals into a **latent space** to extract the underlying structural make of the signal. As the signal passes through the first half, it undergoes a strategic trade-off: **temporal resolution decreases** while **feature depth increases**. This shift effectively moves the data from the raw time domain into an abstract feature domain.

The reconstruction (shown in a lighter blue) is a lossy approximation of the original. Using **Mean Squared Error (MSE)** as an objective often results in a smoother output, as the model prioritises the core signal over high-frequency noise. While this architecture utilises **CNNs**, the same principles apply to **RNNs**, **Transformers**, or **hybrids**.

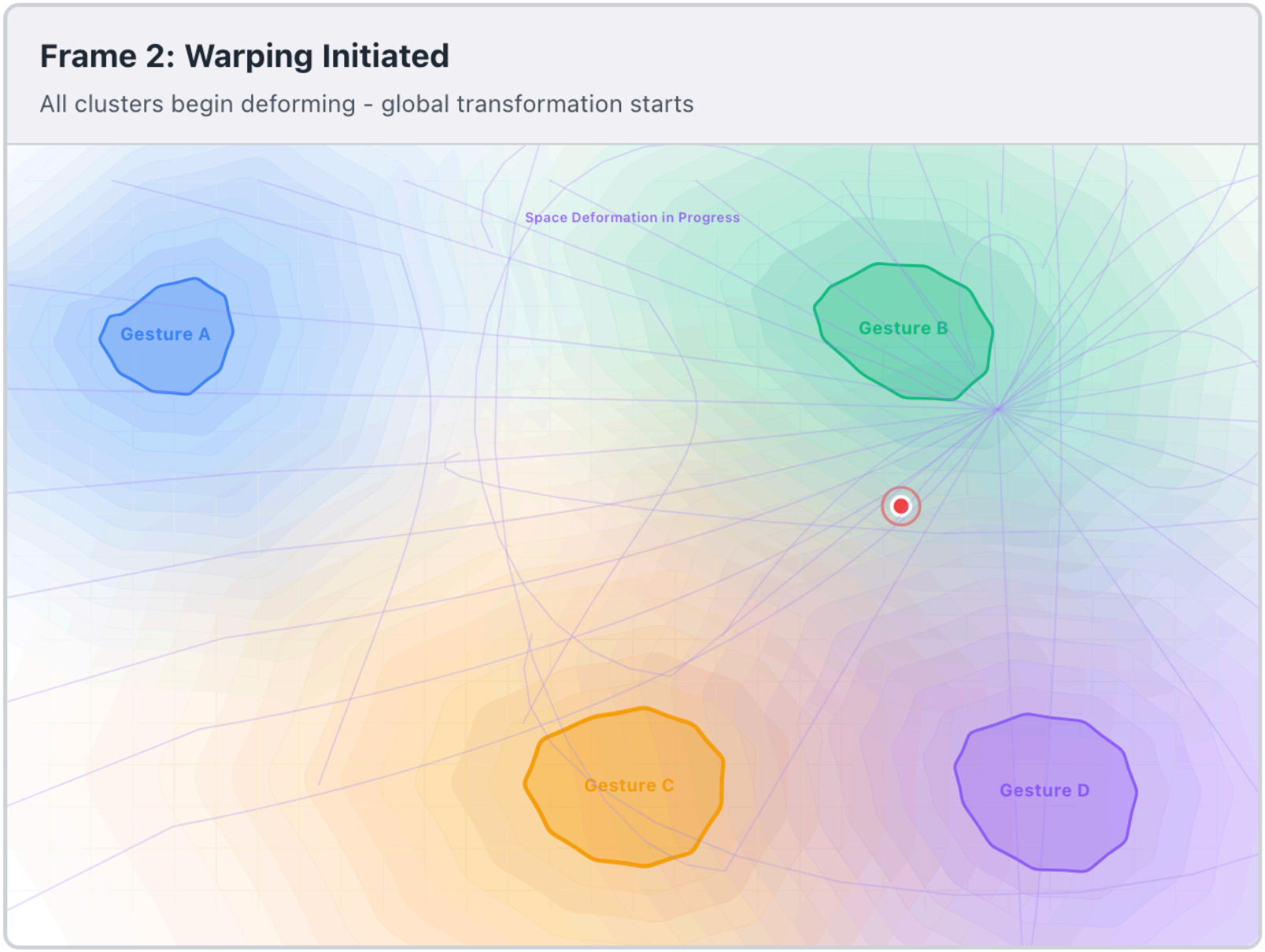
Prior NN, Mathematical Approximation

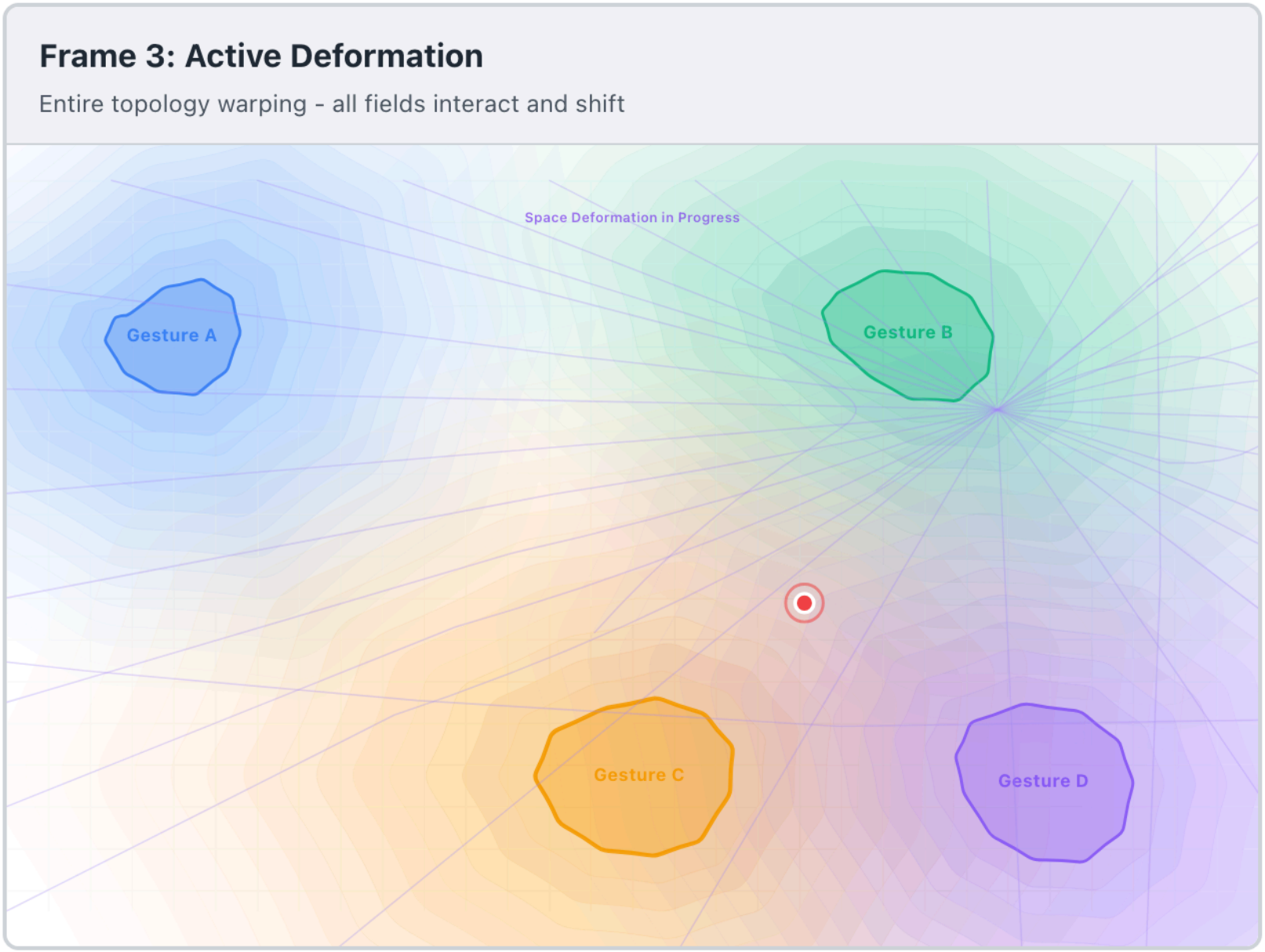


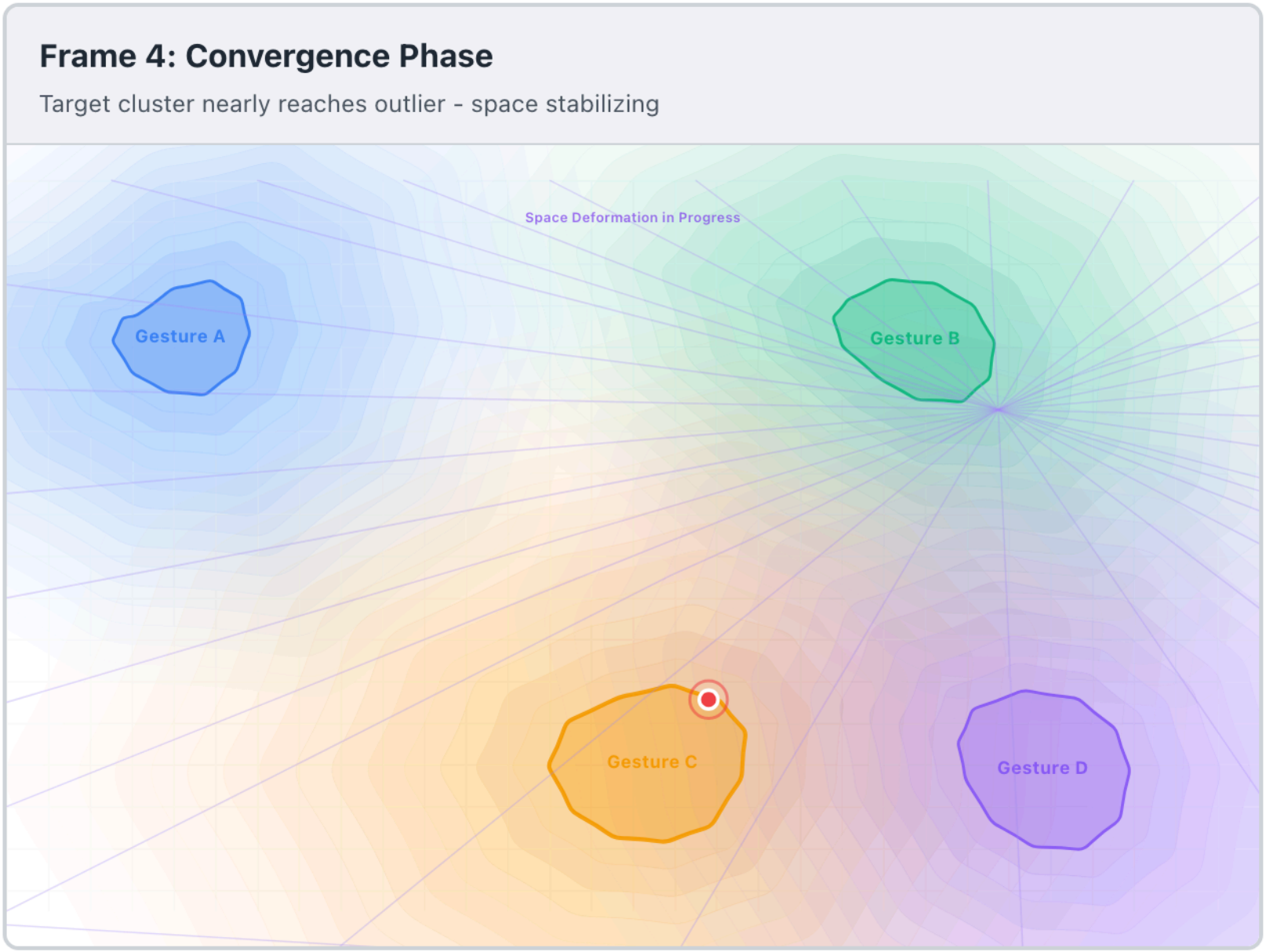


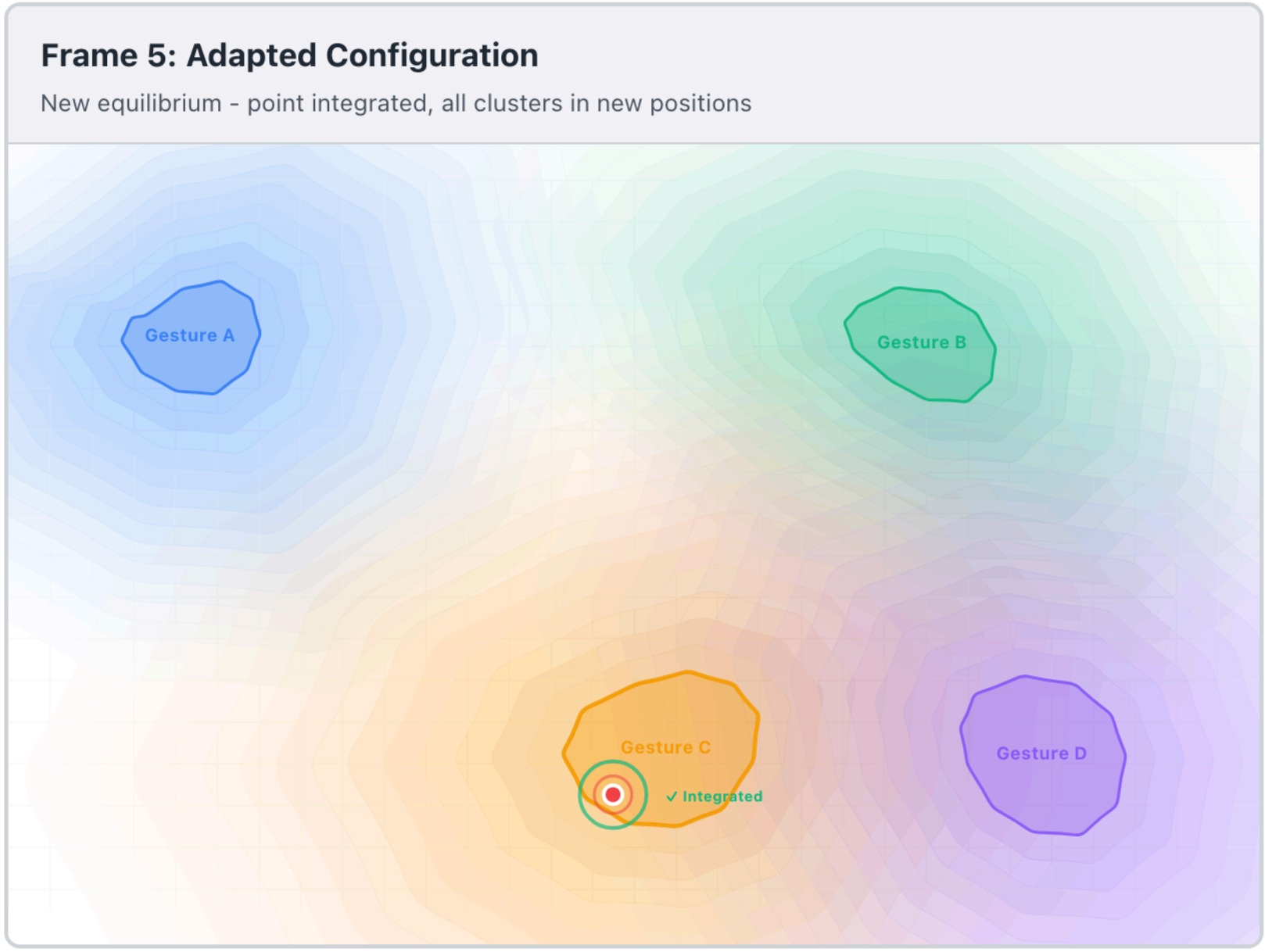
While standard neural networks (MLPs) effectively cluster similar gestures, they operate as static 'black boxes'—opaque, rigid, and requiring complete retraining to learn anything new. To resolve this, I employ a process of **geometric distillation**. By projecting the network's complex behaviour into a **Linear Basis Function model**, I create a transparent 'glass box' surrogate. This approximation creates an interpretable structure that mimics the original model's logic but adds a critical capability: **malleability**. Through recursive updates, this linear framework can instantly adapt to new samples, allowing the system to 'learn' idiosyncratic user nuances in real-time without discarding its foundational knowledge.

Illustration of Approximate Model Updating


Frame 2: Warping Initiated
All clusters begin deforming - global transformation starts
Space Deformation in Progress
Gesture A
Gesture B
Gesture C
Gesture D


Frame 3: Active Deformation
Entire topology warping - all fields interact and shift
Space Deformation in Progress
Gesture A
Gesture B
Gesture C
Gesture D


Frame 4: Convergence Phase
Target cluster nearly reaches outlier - space stabilizing
Space Deformation in Progress
Gesture A
Gesture B
Gesture C
Gesture D


Frame 5: Adapted Configuration
New equilibrium - point integrated, all clusters in new positions
Gesture A
Gesture B
Gesture C
✓ Integrated
Gesture D

FAQs

1. **Is the approximation as accurate as the original?**

   By using Model Distillation, the approximation "shadows" the original network until the error is negligible. While it might lose a tiny fraction of the original's complexity, it gains something more valuable for design: Interpretability. We trade a small amount of "perfect" math for a "workable" structure that a human can actually interact with.

2. **How does the model learn from a new sample without retraining?**

   - It uses **Recursive Least Squares (RLS)**. Instead of looking at the entire history of data every time, the model only looks at:
   - Its current "knowledge" (the Matrix W). Its "certainty" (the Covariance Matrix P).
   - The new sample. It then performs a quick algebraic update that "tweeks" the geometry to accommodate the new information instantly.

3. **What happens if I know a sample is wrong, but I don't know where it should go?**

   This is where the designer intervenes. Since the latent space is a geometric manifold, we can apply a "repulsion force." If a sample is a false positive, we mathematically "push" it away from the cluster it currently sits in. The RLS algorithm then treats this "push" as a new instruction and reshapes the space accordingly.

4. **Does this require the system to be "heavy" or slow?**

   Quite the opposite. Because the math is linear, the updates are nearly instantaneous. This allows for a "fluid" design experience where the latent space feels like a piece of clay that is constantly being refined as the user interacts with it.

5. **How can a simple math equation mimic a complex AI?**

   We use **Linear Basis Function Models**. Think of it like drawing a complex curve by combining many simple shapes (like circles and straight lines). By "expanding" our input features into these different shapes, a simple linear matrix can replicate the sophisticated behaviour of a much larger network.

6. **Why not just use the original Neural Network (the MLP)?**

   Neural Networks are "frozen" once trained. If they make a mistake (a false positive), you can't easily fix it without retraining the whole thing with massive amounts of data. My approach creates a lightweight "student" model that is much more flexible and can be "nudged" or corrected in real-time.